Operating Systems
Project Phase 1 Report

Farah Elnakhal - fqe9080
Myra Rafiq - mr7316

26th February 2026

1 Architecture and Design

The `myshell` program was designed to simulate the basic functionality of a Linux shell by reading user input, parsing it, and executing commands using system calls. The overall architecture follows a modular design to ensure clarity, maintainability, and scalability for future phases.

At a high level, the system is divided into three main components: the main shell loop, simple command execution, and pipeline execution. The main file (`main.c`) acts as the central controller that continuously prompts the user for input, processes the input, and determines how the command should be executed. Depending on whether the input contains a pipe symbol, the shell delegates execution to either the simple command handler or the pipeline handler.

For this phase, the focus was primarily on implementing the functionality for single commands, commands with arguments, and input/output/error redirection. These features are handled in `simple.c`, which encapsulates all logic related to parsing and executing non-pipeline commands.

One important design decision was to preprocess user input before parsing. The function `preprocess_input()` ensures that special symbols such as <, >, 2>, and | are surrounded by spaces. This simplifies tokenization using `strtok()` because it guarantees that each symbol is treated as a separate token. This approach avoids complex parsing logic and reduces potential errors.

```c
void preprocess_input(char *input) {
    char buffer[2048];
    int i = 0, j = 0;

    while (input[i] != '\0') {
        //handle 2>
        if (input[i] == '2' && input[i+1] == '>') {
            buffer[j++] = ' ';
            buffer[j++] = '2';
            buffer[j++] = '>';
            buffer[j++] = ' ';
            i += 2;
        }
        //handle single symbols
        else if (input[i] == '<' || input[i] == '>' || input[i] == '|') {
            buffer[j++] = ' ';
            buffer[j++] = input[i];
            buffer[j++] = ' ';
            i++;
        }
        else {
            buffer[j++] = input[i++];
        }
    }

    buffer[j] = '\0';
    strcpy(input, buffer);
}
```

Figure 1: preprocess_input() function

The program uses arrays of strings (`char *args[]`) to store command arguments. This

aligns with the requirements of the `execvp()` system call, which expects arguments in this format. Additionally, separate variables (`input_file, output_file, and error_file`) are used to store redirection targets, making the execution logic clearer and easier to manage.

The execution model follows the standard Linux shell approach using process creation. For every command, the shell creates a child process using `fork()`. The child process is responsible for executing the command using `execvp()`, while the parent process waits for the child to complete using `wait()`. This ensures that commands are executed sequentially and the shell prompt is only displayed after execution finishes.

Redirection is implemented using file descriptors. The `open()` system call is used to open files, and `dup2()` is used to redirect standard input (`STDIN_FILENO`), standard output (`STDOUT_FILENO`), and standard error (`STDERR_FILENO`) to the appropriate files. This design follows how real Unix shells implement redirection.

The codebase is organized into separate files. `main.c` handles input and control flow, `simple.c` handles simple command execution, and `pipeline.c` handles pipe-related functionality. Header files (`simple.h` and `pipeline.h`) are used to declare functions, ensuring proper separation between modules. A `makefile` is made to automate compilation and enforce consistent build settings.

2 Implementation Highlights

2.1 Command Parsing

The input command is first preprocessed to ensure consistent spacing around special symbols such as `<`, `>`, `2>`, and `|`. This preprocessing simplifies tokenization by guaranteeing that each symbol is treated as a separate token.
After preprocessing, the command is tokenized using `strtok()`, which splits the input string into individual tokens based on spaces. Each token is then analyzed to determine whether it represents:

- A normal argument
- An input redirection operator (`<`)
- An output redirection operator (`>`)
- An error redirection operator (`2>`)

If a token corresponds to a redirection operator, the next token is expected to be a filename. If the filename is missing, an error message is displayed and execution is stopped. Otherwise, the filename is stored in the corresponding variable.
All other tokens are stored in the `args` array, which is later passed to `execvp()` for execution.

```c
// check for input redirection:
if(strcmp(token, "<") == 0){
    token = strtok(NULL, " ");
    //error if missing file after <
    if(token == NULL){
        fprintf(stderr, "Error: Input file not specified\n");
        return;
    }
    input_file = token; // store the input file
}

// check for output redirection:
else if(strcmp(token, ">") == 0){
    token = strtok(NULL, " ");

    //error if missing file after >
    if(token == NULL){
        fprintf(stderr, "Error: Output file not specified\n");
        return;
    }
    output_file = token; // store the output file
}

// check for error redirection:
else if(strcmp(token, "2>") == 0){
    token = strtok(NULL, " ");
    //error if missing file after 2>
    if(token == NULL){
        fprintf(stderr, "Error: Error output file not specified\n");
        return;
    }
    error_file = token;
}

//otherwise, it's a regular argument
else {
    args[arg_count++] = token; // store the token in the args array
}
```

Figure 2: Redirection

This approach ensures that parsing remains simple while still handling important edge cases such as missing filenames.

2.2 Process Creation and Execution

Each command is executed in a separate child process created using the `fork()` system call. This design mirrors the behavior of a real Unix shell, where each command runs independently.

```c
//fork a child process to execute the command
pid_t pid = fork(); // create a new child process
```

Figure 3: Fork

If `fork()` fails, an error message is printed. Otherwise, the program follows two execution paths:

- The child process handles redirection and executes the command
- The parent process waits for the child process to complete

```
else { // parent process
    wait(NULL); // wait for the child process to finish
}
```

Figure 4: Wait

The actual execution of the command is performed using `execvp()`, which replaces the child process with the specified program.

```
// execute the command with its arguments
execvp(args[0], args);
```

Figure 5: execvp

If `execvp()` fails, an error message is displayed and the child process exits to prevent undefined behavior.

2.3 Redirection Implementation

Redirection is implemented using the `open()` and `dup2()` system calls. Each type of redirection is handled as follows:

- Input redirection (`<`) opens the file in read-only mode
- Output redirection (`>`) opens or creates the file and truncates it
- Error redirection (`2>`) redirects standard error output

An example of output redirection is shown below:

```
//handle output redirection
if(output_file != NULL){
    //open the file in write only mode, create it if it doesn't exist, and truncate it if it does exist
    int fd = open(output_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    //error if the file cannot be opened
    if(fd < 0){
        perror("Error opening output file");
        exit(EXIT_FAILURE);
    }

    //redirect standard output to the file
    dup2(fd, STDOUT_FILENO);
    // close the file descriptor after redirection
    close(fd);
}
```

Figure 6: dup2()

The `dup2()` system call replaces the standard file descriptor (such as `STDOUT_FILENO`) with the file descriptor of the opened file. As a result, any output generated by the command is written to the file instead of being displayed on the terminal.

2.4 Error Handling

Several types of errors are handled explicitly:

- **Missing redirection files**
  Example: `ls >`
- **Invalid commands**
  If `execvp()` fails, an error message is printed using `perror()`.
- **File opening errors**
  If a file cannot be opened, `perror()` is used to display the system error message.
- **Empty input**
  The shell ignores empty commands and continues execution.
- **Fork failures**
  If `fork()` fails, an error is printed and execution is stopped.

2.5 Support for Commands with Arguments

Commands with arguments (for example, `ls -l`) are handled naturally by storing all tokens in the `args` array. Since `execvp()` accepts an array of arguments, no additional processing is required beyond proper tokenization. This keeps the implementation simple while supporting a wide range of commands.

2.6 Integration with Main Shell

The main shell loop determines how each command should be executed. If the input contains a pipe symbol (`|`), the command is passed to the pipeline module. Otherwise, it is handled by the simple execution module.

```
//if the input contains a pipe, we will handle it as a pipeline
if(strchr(input, '|') != NULL) {
    execute_pipeline(input);
}
//otherwise, we will handle it as a simple command
else {
    execute_simple_command(input);
}
```

Figure 7: Command type

3 Execution Instructions

The project includes a `makefile` to automate the compilation process. To compile the program, navigate to the project directory and run:

make

This command compiles all source files (main.c, simple.c, and pipeline.c) and generates an executable file named myshell.

```
mr7316@DCLAP-V1525-CSD:~/os_project$ make
gcc -Wall -Wextra -g -c main.c
gcc -Wall -Wextra -g -c pipeline.c
gcc -Wall -Wextra -g -c simple.c
gcc -Wall -Wextra -g -o myshell main.o pipeline.o simple.o
```

Figure 8: Make

Towards the end, if needed, the compiled files can be removed using:

`make clean`

```
mr7316@DCLAP-V1525-CSD:~/os_project$ make clean
rm -f main.o pipeline.o simple.o myshell
```

Figure 9: make clean

After compilation, the shell can be started using:

`./myshell`

Once the program starts, a prompt ($) will be displayed. The user can enter commands just like in a normal Unix shell.

```
mr7316@DCLAP-V1525-CSD:~/os_project$ ./myshell
$
```

Figure 10: myshell

The shell supports:
- Commands without arguments, such as:

`ls`
ps

```
$ ls
'Project Phase 1.pdf'   main.o     myshell      pipeline.h   simple.c   simple.o
 main.c                 makefile   pipeline.c   pipeline.o   simple.h
$ ps
    PID TTY          TIME CMD
2846552 pts/1    00:00:00 bash
2847539 pts/1    00:00:00 myshell
2847957 pts/1    00:00:00 ps
$
```

Figure 11: Commands without arguments

- Commands with arguments:

`ls -l`

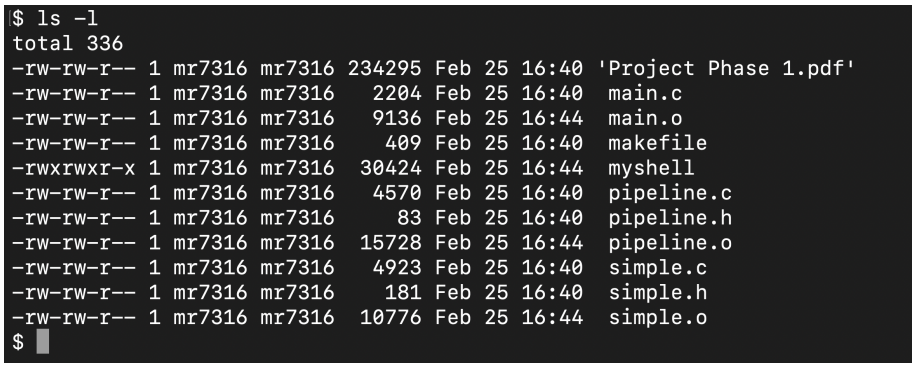

Figure 12: Commands with arguments

- Input redirection:

```
sort < input.txt
```

- Output redirection:

```
ls > output.txt
```

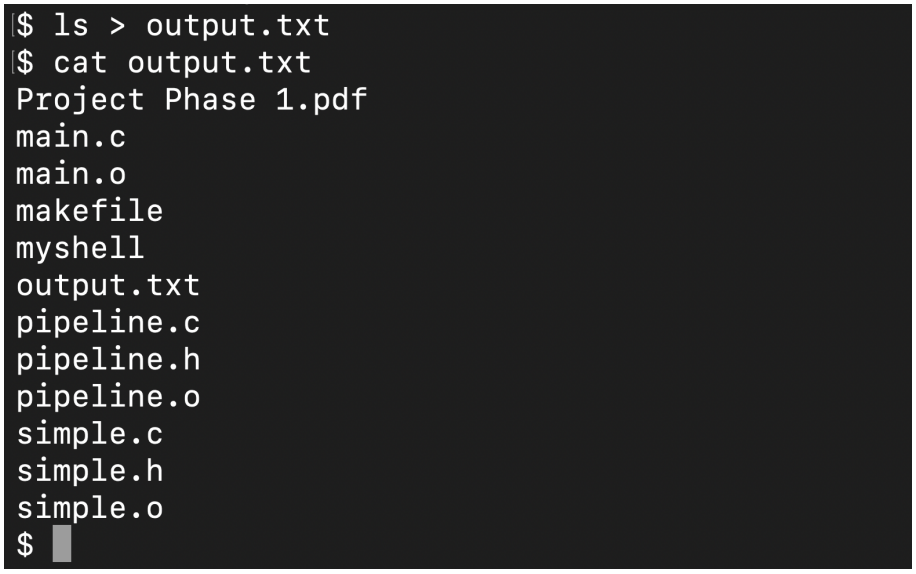

Figure 13: Output redirection

- Error redirection:

```
ls invalidfile 2> error.txt
```

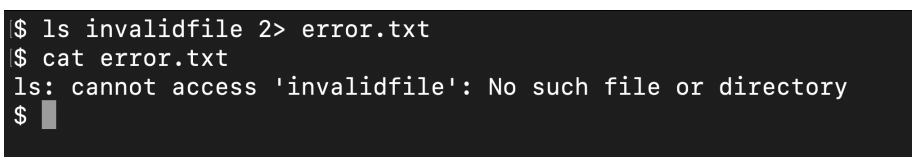

Figure 14: Error redirection

- Combined redirection:

```
sort < input.txt > output.txt
```

To exit the shell, the user can type:

```
exit
```

Figure 15: Exit

The shell follows the standard Unix shell format:

- A $ prompt is displayed
- The user enters a command
- The output is displayed
- The next $ prompt appears

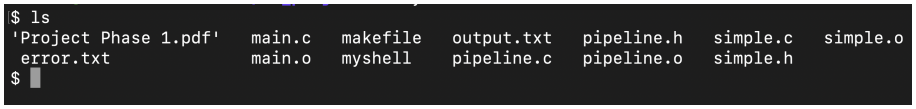

Figure 16: Format

4 Testing

To validate the correctness of `myshell`, a Bash script (`test.sh`) was written to automatically test every feature. Rather than testing commands manually one by one, the Bash file automates the entire process: it sends commands directly into `myshell`, captures the output, compares it against expected results, and prints a clear `[PASS]` or `[FAIL]` for each test case. At the end of the run, a summary line reports the total number of tests passed and failed.

4.1 How to Run the Tests

After the shell is compiled first, place `test.sh` in the same directory as the `myshell` executable. Make it executable and run it using

```
chmod +x test.sh
./test.sh
```

The script handles all setup automatically. It creates a temporary working directory called `test_tmp/`, generates the input files and helper scripts needed for the tests, runs all 58 test cases, and then deletes the temporary directory when finished. No manual preparation is needed.

4.2 How the Bash File Works

The script feeds input into `myshell` by piping commands through `printf`, followed by `exit` to close the shell cleanly after each test:

```
printf '<command>\nexit\n' | ./myshell 2>&1
```

This simulates exactly what a real user would type into the shell. The `2>&1` ensures that both standard output and standard error are captured together, so error messages can be checked as well as normal output.
The script is built around four helper functions that keep each test case concise and consistent:

- `check(name, input, expected)`: runs a command in `myshell` and checks that the output contains the expected string. Used for commands that print directly to the terminal.
- `check_file(name, input, file, expected)`: runs a command and checks that a specific output file was created and contains the expected content. Used for output and combined redirection tests.
- `check_file_nonempty(name, input, file)`: checks that an output file was created and is non-empty, without checking for a specific string. Used for error redirection tests where the exact wording of the error may vary by system.
- `check_absent(name, input, absent)`: checks that a given string does *not* appear in the output. Used to confirm that stderr does not leak to the terminal when it has been redirected to a file.

Each function calls either `pass()` or `fail()` depending on the result. On failure, the expected value and actual output are both printed to make debugging straightforward.

4.3 Test Cases and Results

The 58 test cases are organised into the following categories:

- **Basic commands**: single commands without arguments (`pwd`, `uname`, `ls`), and commands with arguments (`ls -l`, `grep`, `wc -l`, `sort`, `cat`, `head`, `tail`).
- **Output redirection (>)** — redirecting command output to a file, verifying the file contents, and confirming that running the same redirection twice truncates the file rather than appending to it.
- **Input redirection (<)**: feeding the contents of a file to commands via standard input redirection.
- **Error redirection (2>)**: confirming that stderr from failed commands is written to a file and does not appear on the terminal.
- **Combined redirection**: using both < and > together in a single command (`cat < input.txt > output.txt`).

- **Pipes**: single pipe, two-pipe, three-pipe, and four-pipe chains, as well as a large-data test passing 100 lines through a pipe using `seq 1 100 | wc -l`.
- **Composed compound commands**: all combinations specified in the instructoin PDF, including `cmd1 < in | cmd2 > out`, `cmd1 < in | cmd2 | cmd3 > out`, `cmd1 | cmd2 2> err`, and the full combination of input redirection, pipes, and output redirection together.
- **Error handling**: all 8 error messages: missing input file, missing output file, missing error redirection file, missing command after pipe, empty command between pipes, invalid command, invalid command in a pipe sequence, and file not found.
- **Error handling (extra cases)**: pipe at the very start of input, double pipe (`||`), missing output file after `>` inside a pipe, and a bare `>` with no command to confirm the shell does not crash.
- **Shell lifecycle**: `exit` returns code 0, EOF exits cleanly, the `$` prompt is displayed, empty and spaces only input lines do not crash the shell, the shell continues running after a failed command, and multiple sequential commands all execute correctly.
- **Local executables and whitespace**: running a local `./script.sh`, piping from a local script, and confirming that extra whitespace around `|`, `<`, and `>` is handled correctly without breaking execution.

All 58 test cases passed with no failures, as shown in the screenshot below.

```
========================================
 P1 Testing
========================================
[PASS] pwd
[PASS] uname
[PASS] echo with args
[PASS] ls -l
[PASS] cat file
[PASS] grep in file
[PASS] wc -l
[PASS] sort
[PASS] echo > file
[PASS] cat > file
[PASS] > truncates (no append)
[PASS] cat < file
[PASS] grep < file
[PASS] wc -l < file
[PASS] invalid cmd 2> file
[PASS] ls bad path 2> file
[PASS] stderr not on terminal
[PASS] cat < in > out
[PASS] grep < in > out
[PASS] single pipe
[PASS] pipe to grep
[PASS] pipe to wc
[PASS] two pipes
[PASS] three pipes
[PASS] four pipes
[PASS] large pipe: seq 100 | wc -l
[PASS] cmd1 < in | cmd2
[PASS] cmd1 | cmd2 > out
[PASS] cmd < in > out
[PASS] cmd1 < in | cmd2 > out
[PASS] cmd1 < in | cmd2 | cmd3 > out
[PASS] cmd 2> err
[PASS] cmd1 | cmd2 2> err
[PASS] last cmd 2> err in pipe
[PASS] missing input file after <
[PASS] missing output file after >
[PASS] missing error file after 2>
[PASS] missing command after pipe
[PASS] empty command between pipes
[PASS] invalid command
[PASS] invalid cmd in pipe sequence
[PASS] file not found
[PASS] pipe at start
[PASS] double pipe ||
[PASS] missing output after > in pipe
[PASS] bare > no crash
[PASS] exit returns 0
[PASS] EOF exits cleanly
[PASS] $ prompt shown
[PASS] empty line no crash
[PASS] spaces-only no crash
[PASS] shell continues after error
[PASS] sequential commands
[PASS] ./hello.sh
[PASS] ./hello.sh | cat
[PASS] spaces around pipe
[PASS] spaces around <
[PASS] spaces around >

========================================
 Results: 58 passed / 0 failed
========================================
```

Figure 17: Test script output

5 Challenges

One issue encountered early on was what led to the creation of the `preprocess_input()` function, which is responsible for surrounding special symbols with spaces before parsing. Without this function, commands typed without spaces around operators, such as `cat>input.txt`, would fail to parse correctly, as `strtok()` relies on spaces to separate tokens and would treat the entire string as a single token rather than recognising `>` as a redirection operator. The preprocessing step was therefore necessary to guarantee correct tokenization regardless of how the user typed the command. A further issue arose with the detection of `2>`: the original condition only checked whether the current character was `2` followed by `>`, without considering the surrounding context. This meant that any digit `2` immediately followed by `>` anywhere in the input, including inside filenames or command arguments, would be incorrectly treated as an error redirection operator. The fix was to add an additional check ensuring that the character before the `2` is either a space or the start of the string, so that `2>` is only recognised as a redirection operator when it appears as a standalone token.

Getting the `dup2()` and `close()` calls in the correct order inside the pipeline loop was one of the more difficult parts of the implementation. Each child process needs to redirect its stdin to the read end of the previous pipe and its stdout to the write end of the current pipe, and all unused file descriptors must be closed before calling `execvp()`. If any file descriptor was left open, particularly the write end of a pipe, the next process in the chain would block indefinitely waiting for input that never arrives, causing the shell to hang. This was resolved by carefully tracking the previous pipe's read end (`prev_fd`) across loop iterations and ensuring every unused descriptor was closed in both the parent and child after each fork.

6 Division of Tasks

Myra Rafiq was responsible for the basic shell loop, single command execution, argument parsing, input/output/error redirection, and all error handling related to redirection. This includes the full implementation of `simple.c` and `simple.h`.

Farah Elnakhal was responsible for pipe parsing, pipe execution across multiple processes, all pipe and redirection combinations, pipe error handling, and testing. This includes the full implementation of `pipeline.c`, `pipeline.h`, and `test.sh`.

Both members collaborated on `main.c` and their respective parts of the report.